



ANTEPROYECTO DE TRABAJO DE FIN DE GRADO

GRADO EN INGENIERÍA TELEMÁTICA

ESCUELA POLITÉCNICA SUPERIOR

Chatbot inteligente con RAG integrado en simulador logístico (AnyLogistix) para consulta y soporte de simulaciones logísticas

Adrián Encabo Estévez

Tutor: Álvaro Paricio García

Departamento de Automática

2025/2026

1. Introducción

La gestión y optimización de las cadenas de suministro (*Supply Chain*) actuales requieren el análisis de sistemas altamente complejos. Para afrontar este desafío, el uso de Gemelos Digitales (*Digital Twins* [1]) y simuladores de eventos discretos, como el software profesional **AnyLogistix** [2], se ha convertido en un estándar en la industria logística. Estas herramientas permiten modelar redes de distribución, aplicar variabilidad en la demanda y otros parámetros, y evaluar la respuesta de la red frente a escenarios de riesgo. Sin embargo, la operación de estos simuladores y la interpretación de sus resultados exigen un alto nivel de especialización técnica y un manejo manual exhaustivo de bases de datos y configuraciones.

En este proyecto se propone democratizar y agilizar el acceso a estas simulaciones logísticas mediante la integración de Inteligencia Artificial. Específicamente, se desarrollará un agente conversacional inteligente (*chatbot*) a través del cual el usuario podrá interactuar en lenguaje natural para solicitar la ejecución de experimentos y la modificación de escenarios logísticos.

Para lograrlo, el sistema utilizará Modelos de Lenguaje de Gran Escala (*LLMs* [3]) respaldados por una arquitectura **RAG** [4] (*Retrieval-Augmented Generation*) para dotar al agente del contexto necesario. La interfaz conversacional se construirá empleando el *framework* **LangChain** [5] y la librería **Chainlit** [6], todo ello desarrollado en el lenguaje de programación **Python** [7] utilizando **Visual Studio Code** [8] como entorno de desarrollo integrado (*IDE*). El agente traducirá las peticiones del usuario en llamadas estructuradas a la interfaz de programación de aplicaciones (*API RESTful*) de **AnyLogistix**, automatizando la configuración de datos, el lanzamiento de las simulaciones y la extracción de resultados (*KPIs*).

2. Objetivos

El objetivo general de este proyecto es diseñar y desarrollar un agente conversacional inteligente, basado en modelos de lenguaje y arquitectura **RAG**, capaz de integrarse con el simulador **AnyLogistix** para automatizar la configuración, ejecución y análisis de escenarios logísticos a partir de peticiones en lenguaje natural.

Para alcanzar este fin, se establecen los siguientes objetivos específicos (*OE*), ordenados de forma secuencial según el ciclo de vida del proyecto:

OE-1: Diseñar la arquitectura completa del sistema, definiendo el flujo de datos entre la interfaz de usuario, el modelo de lenguaje, el sistema de recuperación de información (**RAG**) y el servidor del simulador.

OE-2: Modelar y configurar un escenario logístico simulable que sirva como base (*Digital Twin*) para plantear el problema de la cadena de suministro sobre el que operará el sistema.

OE-3: Automatizar la comunicación con AnyLogistix desarrollando un cliente en **Python** capaz de consumir su API RESTful para leer bases de datos, lanzar simulaciones de forma remota y extraer resultados estadísticos.

OE-4: Implementar un sistema RAG que permita inyectar contexto técnico y logístico al *LLM*, garantizando que el agente tome decisiones deterministas y precisas basadas en los datos del simulador.

OE-5: Desarrollar una interfaz de *chatbot* interactiva utilizando las librerías **Chainlit** y **LangChain**, permitiendo al usuario solicitar alteraciones en el escenario y recibir el análisis de los experimentos (*KPIs* financieros y de nivel de servicio).

OE-6: Ejecutar y evaluar diversos tipos de experimentos analíticos (tales como Simulación, Variación, Comparación, Stock de Seguridad y Análisis de Riesgos) en **AnyLogistix**, orquestando su lanzamiento directamente desde las directrices procesadas por el *chatbot*.

3. Descripción del trabajo

Se creará una aplicación en **Python** que funcionará como un agente conversacional experto en logística, el cual será accesible a través de una interfaz de chat.

Cuando un usuario interactúe con el sistema para consultar o modificar un escenario, el agente analizará la petición en lenguaje natural, recuperará el contexto necesario de las bases de datos logísticas y de las configuraciones del *Digital Twin*, y orquestrará la ejecución de la tarea. Mediante una arquitectura basada en grafos de decisión, el sistema seleccionará el modelo de lenguaje (*LLM*) óptimo para la tarea, interactuará con la API de **AnyLogistix** para lanzar el experimento correspondiente y devolverá los resultados o *KPIs* procesados al usuario a través del chat.

A continuación, se presenta el esquema general de la arquitectura del sistema:

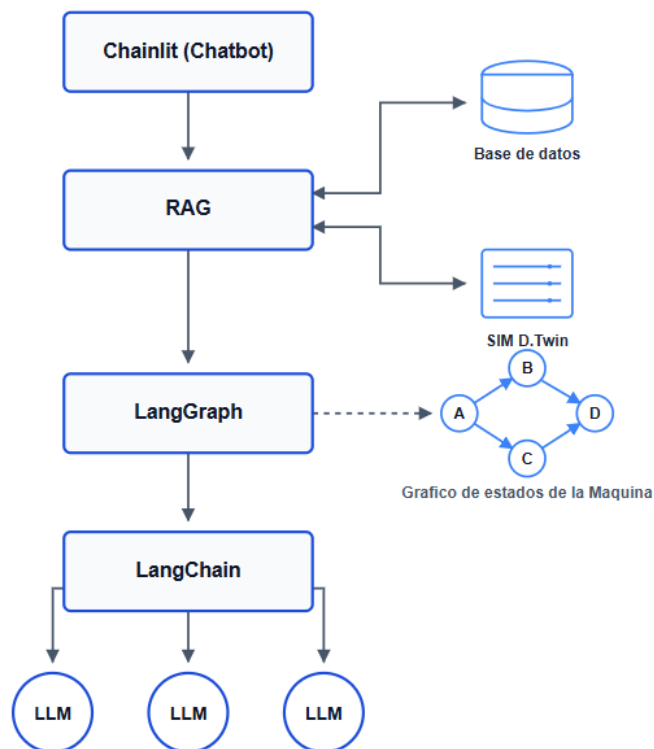


Figura 1. Arquitectura del sistema

¿Cómo se va a hacer?

El enfoque principal será la implementación de un sistema orquestado en capas, donde la lógica de decisión y el enrutamiento de tareas se gestionarán a través de **LangGraph** [9](una extensión de **LangChain**), permitiendo conectar la interfaz conversacional con los modelos generativos y el simulador logístico. La secuencia de actuación del sistema será la siguiente:

1. **Entrada del usuario:** El usuario interactúa con el sistema realizando una consulta o petición en lenguaje natural a través de la interfaz gráfica desplegada con **Chainlit (Chatbot)**.
2. **Aumentación de contexto (RAG):** La petición se procesa a través del módulo **RAG**, el cual se conecta de forma bidireccional con las bases de datos y el simulador *D.Twin*. Este paso permite extraer parámetros actuales de la red, políticas de inventario o resultados previos para dotar al modelo de conocimiento técnico preciso.
3. **Orquestación lógica (LangGraph):** Una vez contextualizada, la petición entra en la capa de **LangGraph**. Esta herramienta define el flujo de ejecución mediante un grafo de nodos (una máquina de estados). Dependiendo de la intención del usuario (por ejemplo, si pide una simulación estándar o un análisis de riesgos), el grafo enruta el proceso por un camino lógico u otro.
4. **Ejecución y enrutamiento (LangChain y LLMs):** La maquinaria de **LangChain** se encarga de conectar la lógica del grafo con múltiples **LLMs**. El sistema puede

disponer de distintos modelos especializados (uno para entender la intención, otro para generar los comandos de la API, otro para redactar el análisis final, etc.).

5. **Interacción con el Simulador:** A través de las funciones construidas, el sistema ejecuta la acción requerida sobre el *Digital Twin* de **AnyLogistix** (modificación de datos, lanzamiento de experimentos, etc.).
6. **Respuesta al usuario:** Los resultados generados o las confirmaciones de las simulaciones viajan de vuelta por la arquitectura hasta ser presentados al usuario en la interfaz del *chatbot*, completando así el ciclo de interacción.

4. Planificación

Para garantizar el cumplimiento de los objetivos descritos y minimizar los riesgos de integración tecnológica, el desarrollo de este proyecto se aborda mediante una metodología iterativa e incremental. El trabajo se estructura en bloques secuenciales donde cada nueva fase añade una capa de complejidad sobre la anterior, culminando en la orquestación del sistema completo.

Las fases de desarrollo establecidas para la consecución del proyecto son las siguientes:

1. Formación inicial

- Estudio de los fundamentos teóricos de la gestión de la cadena de suministro y superación del curso especializado de logística (**EUGLOH** [10]).
- Familiarización con el entorno de simulación **AnyLogistix** y diseño de un escenario base *Digital Twin* simulable.
- Capacitación técnica en el uso de *LLMs*, sistemas **RAG** y *frameworks* de desarrollo (**LangChain**, **LangGraph**, **Chainlit**).

2. Fase 1: Conectividad y Automatización Base (API)

- Configuración del entorno de desarrollo local (**Jupyter Notebooks** [11], [12]**Python**) y gestión de credenciales de seguridad (*API Keys* [13]).
- Desarrollo de un script capaz de interactuar con la *API RESTful* de **AnyLogistix** para recuperar el árbol de proyectos, extraer los identificadores de los escenarios y lanzar simulaciones de forma asíncrona.
- Automatización de la recopilación de resultados (*Dashboards* y *KPIs*) y su exportación en formato tabular.

3. Fase 2: Lógica de Decisión e Interacción de Escenarios (IA Plana)

- Definición de un conjunto de decisiones logísticas predeterminadas.
- Desarrollo de un algoritmo "a prueba de fallos" que manipule los archivos de bases de datos del simulador.

- Automatización del proceso de clonación de escenarios, importación de las modificaciones mediante la API, ejecución del nuevo experimento y exportación comparativa de los resultados.

4. Fase 3: Integración de Modelos de Lenguaje y Arquitectura RAG

- Selección e instanciación del *LLM* que actuará como motor de razonamiento.
- Implementación del módulo **RAG** para inyectar el contexto de la topología de red y las reglas de negocio al modelo, permitiendo que la IA comprenda la semántica de las variables a modificar.

5. Fase 4: Interfaz Conversacional y Orquestación

- Desarrollo de la interfaz de usuario interactiva mediante la librería **Chainlit**.
- Implementación de una máquina de estados utilizando **LangGraph** para enrutar la conversación.

6. Documentación y redacción

- Documentación continua del código fuente y control de versiones a través de un repositorio en **GitHub** [14].
- Redacción progresiva de la memoria académica, análisis de resultados y elaboración de las conclusiones del Trabajo de Fin de Grado.

En la siguiente figura se muestra el Diagrama de Gantt asociado al cronograma del proyecto:

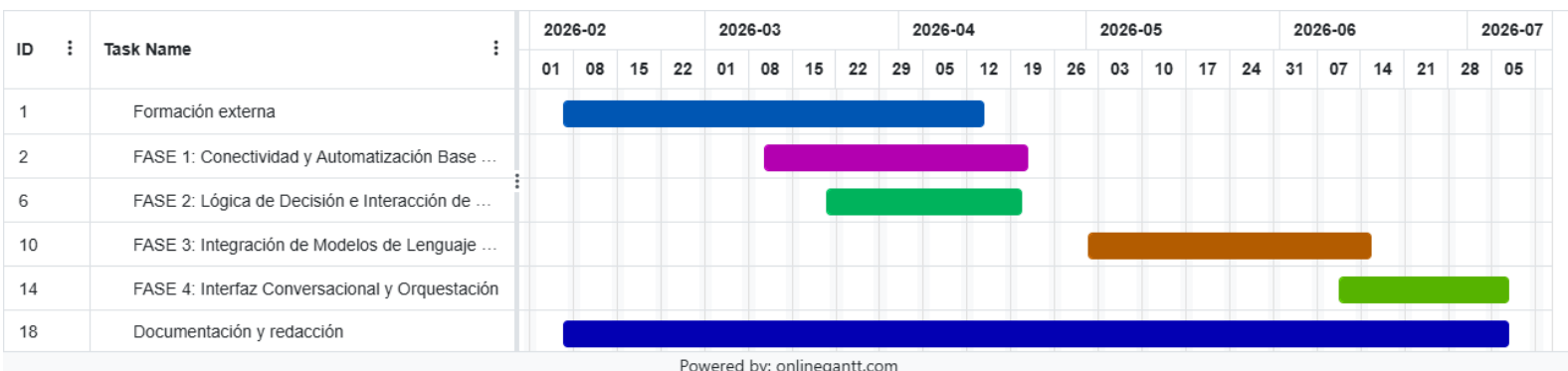


Figura 2. Cronograma del proyecto

5. Herramientas utilizadas

Las herramientas que van a ser necesarias para desarrollar este proyecto son las siguientes:

- Un ordenador personal con capacidades para ejecutar el sistema desarrollado.
- Software y plataformas:

- Lenguaje de programación: **Python**.
- Simulación logística: Servidor de **AnyLogistix** y su API RESTful.
- Orquestación de agentes e IA: **LangChain**.
- Interfaz de usuario: **Chainlit**.
- Entorno de desarrollo: **Visual Studio Code**, **Anaconda** y **Jupyter Notebook**.
- Gestión de datos y soporte: **Microsoft Excel** y librería **Antigravity** [15].
- Control de versiones: **GitHub**.

Bibliografía

- [1] «¿Qué es un gemelo digital? | IBM». Accedido: 25 de abril de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/digital-twin>
- [2] «anyLogistix: Supply Chain Optimization, Simulation & Design Software Tool». Accedido: 24 de abril de 2026. [En línea]. Disponible en: <https://www.anylogistix.com/>
- [3] C. Stryker, «¿Qué son los grandes modelos de lenguaje (LLM)? | IBM». Accedido: 26 de abril de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/large-language-models>
- [4] «RAG - Retrieval Augmented Generation: La llave que abre la puerta de la precisión a los modelos del lenguaje | datos.gob.es». Accedido: 24 de abril de 2026. [En línea]. Disponible en: <https://datos.gob.es/es/blog/rag-retrieval-augmented-generation-la-llave-que-abre-la-puerta-de-la-precision-los-modelos-del>
- [5] «LangChain: Observe, Evaluate, and Deploy Reliable AI Agents». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://www.langchain.com/>
- [6] «Chainlit - Build AI applications». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://chainlit.io/>
- [7] J. VanderPlas, «Python Data Science Handbook».
- [8] «Visual Studio Code - The open source AI code editor | Your home for multi-agent development». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://code.visualstudio.com/>
- [9] «LangGraph: Agent Orchestration Framework for Reliable AI Agents». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://www.langchain.com/langgraph>
- [10] «Eugloh - eugloh.eu», Eugloh. Accedido: 21 de abril de 2026. [En línea]. Disponible en: <https://www.eugloh.eu/>
- [11] «Project Jupyter». Accedido: 26 de abril de 2026. [En línea]. Disponible en: <https://jupyter.org>
- [12] «¿Qué es Jupyter Notebook? | Databricks». Accedido: 26 de abril de 2026. [En línea]. Disponible en: <https://www.databricks.com/es/blog/what-is-jupyter-notebook>
- [13] «¿Qué es una clave API? | IBM». Accedido: 26 de abril de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/api-key>
- [14] «Build software better, together», GitHub. Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://github.com>
- [15] «Google Antigravity». Accedido: 1 de mayo de 2026. [En línea]. Disponible en: <https://antigravity.google/>